

Server-side scripting with PHP4

Michael Schacht Hansen (msh@hi.au.dk)
Lars Riisgaard Ribe (lars.ribe@iekf.au.dk)
Section for Health Informatics
Faculty of Health Sciences
University of Aarhus
Denmark

June 19, 2001

Contents

1	Preface	3
1.1	What is PHP?	3
1.2	What is this document?	3
1.3	Prerequisites	3
2	Introduction	3
2.1	What is server-side scripting	3
2.1.1	Standard static web pages	3
2.1.2	Web pages generated with server side scripting	4
2.2	When to use server-side scripting	5
2.3	What you need to get started	5
3	Installation	6
3.1	Linux server	6
3.1.1	Selecting a Linux distribution	6
3.1.2	Installing Linux	7
3.1.3	Installing packages after Linux installation	7
3.1.4	Testing the web server	7
3.1.5	Test the PHP installation	8
3.1.6	Test the MySQL installation	8
3.2	Windows 2000 server	9
3.2.1	Installing MySQL in Windows 2000	9
3.2.2	Installing PHP in Windows 2000	9
3.2.3	Using Microsoft Access databases with PHP	10
4	The first PHP application	10
4.1	Application description	10
4.2	The Form	11
4.3	The PHP script	12
4.4	Testing the application	12
5	Programming with PHP	13
5.1	Basic syntax	13
5.1.1	Escaping HTML	13
5.1.2	Instruction separation	14
5.1.3	Comments	14
5.2	Variables and Data types	14
5.2.1	Basic variable syntax	14
5.2.2	Data types	15
5.2.3	Variables passed from HTML forms	16
5.3	Control structures	17
5.4	Functions	19
5.5	Getting more information on the PHP language	20

6	Interfacing databases	20
6.1	Structured Query Language SQL	20
6.1.1	SELECT command	20
6.1.2	INSERT command	22
6.1.3	UPDATE command	22
6.2	Scenario: MySQL database	22
6.2.1	Creating a database and a database user	23
6.2.2	Extracting information from the database with PHP . . .	25
6.2.3	Creating new records with PHP	26
6.2.4	Updating records with PHP	27
6.2.5	Finishing the application	29
6.3	Scenario: Access database on Microsoft Windows 2000 Server . .	30
6.3.1	Extracting information from the Microsoft Access database with PHP	31
6.3.2	Creating new records with PHP	32
6.3.3	Updating records with PHP	33
6.3.4	Finishing the application	35

1 Preface

1.1 What is PHP?

PHP (originally “Personal Home Page”) is a programming language used to enhance web pages. It is a server-side scripting language meaning, that it is executed on the server as apposed to client-side scripting languages like Javascript. PHP is also an embedded scripting language. This means that you can design your web pages as usual and embed the programming language where it is needed. PHP is very often used to extract dynamic web content from a database.

1.2 What is this document?

This document is a brief introduction to server-side scripting with the PHP scripting language. Focus is on the general principles of server-side scripting and the PHP language has been chosen as an example. For a more complete manual on PHP, consult the PHP reference manual [1].

1.3 Prerequisites

It is assumed that the reader has some prior experience with web design and some basic understanding of HTML. Experience with WYSIWYG¹ HTML editors is generally not enough, since you have to wite a lot of HTML by hand when doing server-side scripting. For an introduction to HTML refer to “A short introduction to HTML and CSS” [2].

2 Introduction

In this section we will focus on the basic principles of server-side scripting. In what ways does server-side scripting differ from standard static web pages?

2.1 What is server-side scripting

2.1.1 Standard static web pages

Many of the pages that we view on the web are static. This means that they don’t change, once the author has finished writing/designing them. When a user types in the address of a specific page in a web browser, the communication follows a model similar to figure 1. A request is sent to the server. The request contains the name of the requested document and several header fields. The header fields contain information about the browser requesting the document. The server locates the document on the disk and returns it to the user’s browser. The browser then displays the document according to the settings and specifications of the browser. The browser might also respond to some of the header fields. It might store a cookie on the users machine or it might be redirected to another web page. Normally the web author has no control over these header fields when designing web pages, but they can be manipulated by server-side scripting as we shall see.

¹What You See Is What You Get

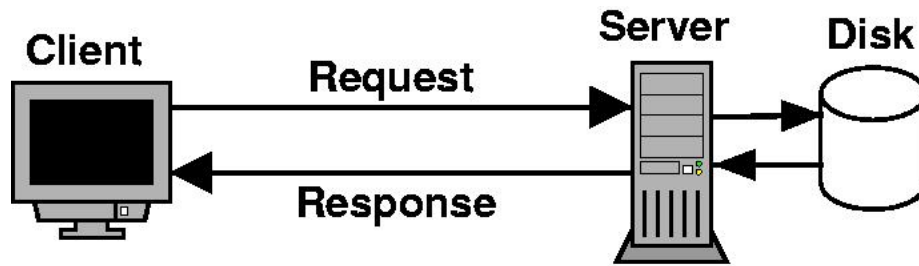


Figure 1: Communication model for static web pages.

2.1.2 Web pages generated with server side scripting

When a web server serves pages generated with server side scripting, the communication model becomes a bit more complicated. Figure 2 illustrates this communication. The pages referred to in the address field of the users browser is no longer an HTML file (or another static resource like an image). It is a script² and the web server calls up an interpreter to execute the script. As indicated in

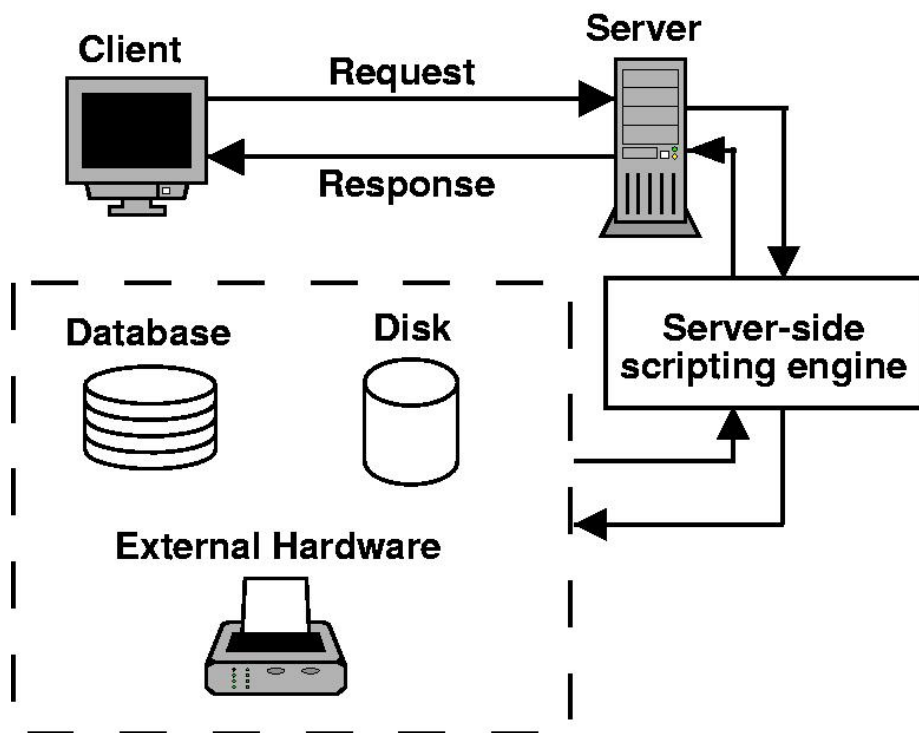


Figure 2: Communication model for dynamic web pages generated with server-side scripting.

²A URL can also refer to a compiled program. In that case you would not call it server-side scripting (since it is not a script), but the communication model would still be the same.

figure 2 the scripting engine is often in contact with several resources. This connection to databases or external hardware is what makes server-side scripting so attractive. The scripting engine can make decisions based on databases or other resources and generate web pages that change dynamically. The scripting engine can also make decisions based on user input, making it possible to create web sites that are more interactive.

2.2 When to use server-side scripting

Once you get familiar with a sever-side scripting language you notice many situations where you might benefit from using scripting. You might even find that you use it on every single web page you create. Scripting does however give the server an extra workload and you should consider if it is really necessary in all the situations where you use it. The obvious situations to use server-side scripting fall into three categories (they are somewhat overlapping, but still different situations):

- Interfacing databases. Whenever you have a database and you want to create a web-interface for it, you need scripting. It might be a database of customers, a searchable image database etc.
- Controlling or monitoring external hardware. Process control and monitoring through web-interfaces is becoming very popular. You might control the heating system of your house or in the future monitor how critically ill patients are doing.
- Content management systems. If you are creating a web site and someone else is supplying the contents for that web site. Especially if that someone doesn't know how to create web pages. You might setup a system for uploading and storing the contents and generate the web pages on the fly from a database.

2.3 What you need to get started

Hopefully you have established a basic idea of what server-side scripting is, and when you should use it. Next you need some basic requisites to be able to write and execute scripts. Basically you need (for installation details refer to section 3):

1. First of all you need a computer with a web server installed. PHP will run on Windows NT/2000, most Unix type operating systems (Linux, Solaris, Irix etc.) and MacOS X.
2. Then you need to have PHP (preferably version 4) installed. PHP is available for free.
3. If you want to interface a database you should install a database engine. You may use a Microsoft Access Database on the Windows platform for, but we recommend getting a database like Microsoft SQL Server, MySQL (this one is available for free), or Oracle if you are doing larger projects.

4. You also need some sort of text editor. Almost any text editor will do, we recommend a text editor like Emacs [3] or Ultra Edit [4]. It is important to get comfortable with your text editor. You will be spending a lot of time with it. You could also use whatever HTML editor you are using at the moment, but some of them get confused (and report errors) when you start putting PHP (or any other scripting language) in your HTML documents.

3 Installation

Your installation scenario depends on your operating system. Here we will go through some of the considerations for two different scenarios:

1. A Linux operating system running Apache web server, MySQL and PHP.
2. A Windows 2000 operating system running Internet Information Server (IIS), MySQL/Access database and PHP.

3.1 Linux server

The Linux solution can be installed without paying any license fees (it is completely free), and interestingly enough is probably the easiest solution to install since all the components you need are included with all major Linux distributions. If you start rebuilding (compiling) new versions of the necessary components things get a little more complicated, but you should not need to do that unless you have very specific needs.

3.1.1 Selecting a Linux distribution

The first thing you need to do is select a Linux distribution. A distribution is a collection of software containing the core operating system (the kernel) and several programs that run under the operating system (for instance a web server). There are many different distributions of Linux. For the present purpose you can choose any of the following:

- RedHat. Current version is 7.1
This is probably the most popular distribution. Easy to install and everything you need is there.
www.redhat.com
- Mandrake. Current version is 8.0
A lot like RedHat, but maybe a bit more user-friendly and easier to install.
www.linux-mandrake.com
- SuSE. Current version is 7.2
Use this if you want a large distribution with everything in it. You should order their CD-set.
www.suse.com
- Debian. Current version is 2.2r3
Not as user friendly as the others. More a developers/nerd platform.
www.debian.org

If you have no idea which to choose, we recommend you download RedHat or Mandrake from their web sites and burn installation CD-ROMs. The installation CD-ROMs are usually bootable, meaning you can insert them in the CD-ROM drive and reboot the computer to start installation.

3.1.2 Installing Linux

This document is not an installation guide to Linux, and you should consult the document on the distribution that you have downloaded. In this section we will make sure that you remember to install the components needed for running PHP and a web server.

The installation programs usually have an automatic mode, which does all the harddrive partitioning etc. for you. Be careful with those wizards if you have data on your hard drive, that you don't want to loose. In general you should not experiment on computers that are "mission critical".

Once the installation program gets started you have to choose which packages to install. Linux distributions consist of many different software packages³ You need to install the following packages to be able to use PHP and databases:

1. Apache web server. This is the standard web server on Unix type computers.
2. mod_php. This is a plug-in module to allow Apache to execute PHP scripts.
3. MySQL. The database server.
4. MySQL client. Console based client program for manipulating your database.

The Linux distribution might need to include other packages, to solve dependencies, but this is usually solved automatically. There might also be a possibility of doing a server (or web server) type installation. This type of installation usually includes the required components and configures them for you.

3.1.3 Installing packages after Linux installation

If you already have a Linux box running you will need to have the packages mentioned above. These packages can usually be found on the installation CD-ROM or on the internet. I recommend <http://rpmfind.net> (Debian does not use rpm packages) if you are looking for packages for your distribution. RPM packages are easy to install on Linux distribution. You can use commands like:

```
rpm -ivh packagename.rpm
```

The RPM installation program checks if you have the necessary packages required by the package you are installing and tells you if any are missing.

3.1.4 Testing the web server

The web server has a root directory somewhere, where it keeps the HTML files. The location of this root directory varies from distribution to distribution. Usually the individual users on the server also have rights for creating web pages in their own directory. These web pages are usually located in a folder called:

³Many distributions use the Redhat Package Manager (RPM) packages.

```
/home/your_user_name/public_html
```

If this directory does not exist in your home directory you should create it with the command:

```
mkdir /home/your_user_name/public_html
```

Try placing an HTML file in that directory and type the the following address in your browser:

```
http://server_name/~your_username/name_of_file.html
```

You should see the HTML document displayed in the browser.

3.1.5 Test the PHP installation

Create a document called `test.php` containing the following:

```
<html>

<body>

<?phpinfo() ?>

</body>

</html>
```

Type the address of the new document in your browser and you should see a PHP info page.

3.1.6 Test the MySQL installation

You can check if MySQL is running by trying to log into it. This is done with the following command:

```
mysql -u root -p
```

The server will prompt you for a password. If you are running a new installation, the password is blank and you can press enter. If MySQL is running you should see something like:

```
mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2 to server version: 3.23.36
```

Type 'help;' or '\h' for help. Type '\c' to clear the buffer

```
mysql>
```

You could try issuing a command like:

```
mysql> SHOW DATABASES;
```

This should print a list of the available databases.

3.2 Windows 2000 server

This guide is based on a Windows 2000 Server running Internet Information Services (IIS), which is the default web server. If you are using another web server than IIS in a Windows 2000 Server, the installation might work automatically, but after the installation has finished you can otherwise refer to `install.txt` and `readme.txt` in `C:\php` for information about integrating PHP with your server.

If you would like to use MySQL as your database server, MySQL should be installed first. For installation of MySQL please read section 3.2.1. If you are not interested in using MySQL, you can go straight to the installation of PHP in section 3.2.2.

3.2.1 Installing MySQL in Windows 2000

First of all you have to download MySQL from www.mysql.com. The latest stable version is currently 3.23. During the installation you should use `c:\mysql` as the directory in which MySQL should be installed, otherwise it doesn't work. Once the installation has finished the program is on the computer, but the MySQL server is not up and running yet. First of all, MySQL has to be registered as a service, and second the MySQL server has to be started. These steps are accomplished by opening an MS-DOS-box and type:

```
C:\mysql\bin\mysqld-nt --install <ENTER>
NET START mysql <ENTER>
```

Now the MySQL server is up and running. For information about testing MySQL, please refer to section 3.1.6.

3.2.2 Installing PHP in Windows 2000

PHP can be installed using an installation program with standard features, it can be manually put together and it can be build from the ground. It is absolutely recommended to use the version with the installation program, since it does all the configuring for you. If, on the other hand, you run another web server than IIS and the installation program doesn't work for you, you might put it together manually. Please refer to the documentation at www.php.net for further details (the rest of this section is based on the installation program).

First of all, you have to download PHP from www.php.net. The current stable version is 4.0.5, and you should select the version with "packaged as Windows installer" (the small one). Before running the installation program, make sure that you are administrator and you have `MSCOMCTL.OCX` in `C:\winnt\system32`. This file is needed for the automated installation. If the file isn't there, you will have to get it from somewhere else; the file comes with MSOffice, MS Visual Studio and other Microsoft programs. You can simply copy the file from a computer that has it to `c:\winnt\system32` and run the following command:

```
regsvr32 mscomctl.ocx
```

You are now ready to run the installation program⁴. When the program asks for

⁴If you haven't got access to `MSCOMCTL.OCX`, it might be possible to install without it. Simply install, click OK that you are missing `MSCOMCTL.OCX` and then read in the IIS 4+ section in `install.txt` in the PHP directory for more details

the SMTP server, you can fill it in if you know it or just leave as it is. Also leave the selection of Microsoft IIS 4 or higher. When asked for the IIS Scriptmap Node Selection mark all topics, if you want to run PHP on all web servers.

After the installation it is recommended, that the `IUSR` is granted **read** and **execute** rights to `php.exe` and `php4ts.dll`, and **list folder content**, **read** and **write** rights to the sub folders of `c:\php`.

Testing the PHP server is done as in section 3.1.5.

3.2.3 Using Microsoft Access databases with PHP

Access to a Microsoft Access database is through a common database-interface called ODBC. It is not necessary to install any general programs to use a Microsoft Access database in PHP, but every Microsoft Access database that will be accessed through PHP need to be registered as an ODBC source. To select the right database when making database calls, every ODBC source must have a name: the DSN.

The setup of the database type, the DSN and the actual filename is done in **Data Sources (ODBC)**. This program is found in:

Start>Programs>Administrative tools.

In **systemDSN** you can add an entry for your database. Here you select the type (select Access database in english), the DSN(the name you use in PHP to access it), and the file name. It is possible both to create a new database or use an existing one.If you create a new database it will be empty so you will need to create tables and columns in the database. This can be done either by using Microsoft Access or by using SQL commands through PHP. Remember to give the `IUSR` read and write access to the database file.

After this is complete, you can access your database through an ODBC call to the name you specified as the DSN.

4 The first PHP application

Before we start getting into the gory details of the PHP programming language we will take a look at a very simple example application. Use this application to get a basic idea of PHP and to test if your server is setup correctly.

4.1 Application description

This first test application consists of a form (an HTML document) for data input and a PHP script for interpreting the data input and displaying a result in the browser window. The application asks for an email address and does some processing on the text entered in the field. It parses the text and locates the mailbox (mailbox alias) and the server name⁵. It is assumed that the user enters an email address, but there is no data validation. This is not a useful approach for a real life application, but it will do for this test.

⁵They are separated by the “@” in the email address

4.2 The Form

The form is very straight forward. Open your text editor and type in the following.

```
<html>
<head>
    <title>Simple Test Application</title>
</head>

<body>
<h1>Simple Test Application</h1>
<form action="do_form.php" method="post">

Type your email address:<br>
<input type="text" name="email">
<p>
<input type="submit">

</form>
</body>

</html>
```

Save the file with the extension “*.html” or “*.htm”. The form should look like figure 3.

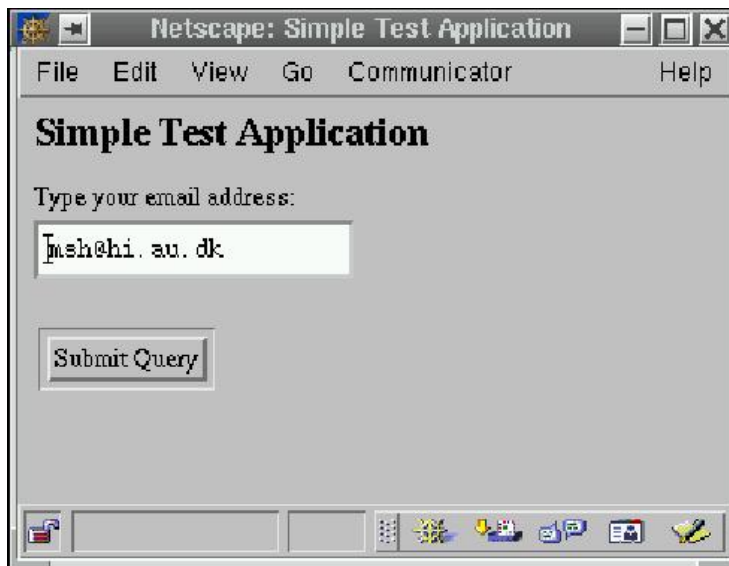


Figure 3: A simple form for entering an email address.

4.3 The PHP script

The next thing we need is a script for interpreting the information entered in the form. Enter the following in your text editor and save the file as `do_form.php`. The `action` property of the `form` tag in the HTML file above states that the information from the form should be sent to a file called `do_form.php`.

```
<html>
<head>
<title>Simple Test Response</title>
</head>

<body>
<h1>Response from server</h1>

<?php
echo "Your email address is: <b>$email</b> <br> \n";
list ($mailbox, $server) = split ("@", $email, 2);
echo "Your email must be located at the server <b>$server</b>.<br> \n";
echo "And your mailbox is called: <b>$mailbox</b> <br> \n";
?>
</body>

</html>
```

4.4 Testing the application

Your first PHP application should now be ready to run. Place the two files (the form and the PHP script) in a folder on the web server, and open a browser. Type the address of the form in the address field, fill in the form and hit submit. The output should look like figure 4.



Figure 4: Output from the first, simple PHP application.

This first example application illustrates one of the ways PHP can be used to make you web pages interactive. You should not worry if you don't understand

every single line of code in the example. The important points to note in the example are:

- Forms can be used to send information to scripts. The `action` property of the `form` tag indicates where the information should be sent.
- PHP scripts are text files that look much like HTML, but a special tag `<?php ?>` escapes the HTML mode and allows the developer to write code in the PHP scripting language.
- The PHP scripting language has commands for processing input from forms and writing output to the user's browser.

5 Programming with PHP

This section contains some of the boring details about the PHP scripting language. You can read this section now or use it as a reference later. If you are new to programming we recommend you read it now.

5.1 Basic syntax

5.1.1 Escaping HTML

PHP is an embedded scripting language. This means that your PHP scripts are really just HTML documents with PHP added where it is needed. You need to escape the HTML mode to write PHP. There are several ways of doing this. The following example illustrates the standard way of escaping HTML:

```
<html>

<head>
<title>How to escape HTML mode</title>
</head>

</body>
<h1>How to escape HTML mode</h1>

<?php
    echo "We are now in PHP mode<br>\n";
?>

This is standard HTML mode.<br>

</body>

</html>
```

There are several other ways of escaping HTML (listed below) but I recommend the approach illustrated above.

```
<?php echo("Use this approach\n"); ?>
```

```
<? echo ("Simple version\n"); ?>
```

```
<script language="php">  
echo ("For editors like FrontPage");  
</script>
```

```
<% echo ("ASP-style tags"); %>
```

```
<%= $variable; %>
```

5.1.2 Instruction separation

Like any other programming language PHP consists of separate instructions. These instructions need to be separated with semicolons (;). Several instructions can be written on one line as long as they are separated with semicolons. On the other hand you are allowed to spread one instruction over several lines. This is illustrated below:

```
<?php  
echo "This"; echo "is"; echo "several"; echo "instructions";  
  
echo "This, on  
the other  
hand, is only  
one instruction";  
?>
```

5.1.3 Comments

Comments can (and should be) placed in the PHP code using one of the following notations.

```
//C++ Style one-line comment
```

```
# Shell style one-line comment
```

```
/* Multi  
Line  
Comment */
```

We recommend that you adapt one of the styles and stick with it to avoid any confusion.

5.2 Variables and Data types

5.2.1 Basic variable syntax

Variables are indicated by a dollar sign (\$) followed by the variable name. Variable names may contain letters or numbers and the underscore character (_).

Variable names must begin with a letter or the underscore character. Variable names are case-sensitive. The following lines illustrate valid variable names:

```
$firstname = "Michael";
$firstName = "John";
$first_name = "Peter";
$age = 23;
```

A variable is defined when a value is assigned to it. Variables are valid within the context in which they are defined (variable scope). This usually means within the entire PHP script. The variables are however not automatically valid within functions (refer to section 5.4). Variables within functions are only defined in the functions (local variables) and global variables needed in the functions must be explicitly declared with the `global` keyword.

5.2.2 Data types

There are 4 scalar types (booleans, integers, floating point values and strings) in the PHP language. The usage of these types is straight forward. The example below illustrates assignment of the scalar types:

```
$theTruth = TRUE; //Value can be TRUE or FALSE
$age = 23;
$income = 0.0034;
$firstname = "Michael";
$lastname = 'Hansen';
```

There are also two compound types in PHP: objects and arrays. Objects are used in Object Orientated Programming; a rather advanced technique, that you will not use at first. Arrays are, on the other hand, important since you will be using them heavily when interfacing databases (probably the number one reason to use server-side scripting). Arrays are variables that contain more than one value. The individual variables are accessed through indexes or keys. It may sound abstract, but it is really quite simple: Imagine that a variable is a book with several pages. The pages can either be numbered (starting with page 0) or labeled with text labels (or both). Statements like “What is on page 2” or “Set the content of page 3 to the number 45” would make sense. If the pages were labeled you could state “What is on the page labeled Introduction”. In PHP the syntax for manipulating arrays is implemented as illustrated in the following examples:

```
<?php
```

```
//Creating an array with the array language construct
$a = array("This","is","an","array");
```

```
echo $a[2]; //prints "an"
//The first value in the array has index 0
```

```
//Arrays with keys
$b = array( 'color' => 'red',
           'taste' => 'sweet',
```

```

        'shape' => 'round',
        'name'  => 'apple');

echo $b['taste']; //prints "sweet"

//Creating arrays by assigning values
$c['color'] = 'red';
$c['taste'] = 'sweet';
$c['shape'] = 'round';
$c['name'] = 'apple';

echo $c['taste']; //Also prints "sweet"

//Assigning values through indexes
$d[0] = "Another";
$d[1] = "Test";

echo $d[1]; //Prints "Test"

?>

```

PHP does not support explicit type declaration. The type of a variable depends on the context in which it is evaluated. If you try to output a variable (e.g. with `echo`) it is evaluated as a string. If you try to add an integer to it, it is evaluated as an integer and so on. This makes it a little easier to get started with the language, because you don't need to worry about data types. On the other hand it is left up to the programmer to keep track of data types. This can cause unpredicted behavior if you are not careful.

5.2.3 Variables passed from HTML forms

An important discipline in server side scripting is passing information from forms to PHP scripts for further processing. The simple application above (section 4) is an example of this. The passing of information from forms to scripts is very smoothly integrated in PHP. Consider the form created with the following HTML code:

```

<form action="script.php" method="post">
    First Name:<br>
    <input type="text" name="first">
    <p>
    Last Name:<br>
    <input type="text" name="last">
    <p>
    <input type="submit">
</form>

```

This form creates two input boxes (named `first` and `last`) and a submit button. When the submit button is clicked the contents of the form is sent to the server. To be more precise the contents is sent to be processed by a script called `script.php`. Lets look at the script `script.php`:

```

<html>

<head>
    <title>Processing script</title>
</head>

<body>
<h1>Output</h1>
<?php
    echo "Last name: ";
    echo $last . "<br>";

    echo "First name: $first";

?>
</body>

</html>

```

This little example illustrates the most important features for passing information from forms to scripts (but also a few general features). You should notice the following:

- Variables with the same names as the names of the input boxes are automatically generated for you. When you have an input box named **first** in your HTML document, the contents of this input box will be available to you in a variable named **\$first** in the PHP script receiving the content of the form.
- Information can be written in web pages using the **echo** function (**print** would have the same effect).
- Strings are indicated by quotation marks.
- Strings are concatenated (glued together) with the **.** operator.
- Variable names within double quotation marks are evaluated before printing the string. Had we used single quotation marks the variable name would have been printed instead of the value (try replacing the double quotation marks with single quotation marks).

You will be using string manipulation and forms heavily in your work with PHP and I recommend that you consult the PHP manual frequently for information on special string manipulation functions.

5.3 Control structures

All programming languages have a series of control structures for decision making and controlling program flow. Instead of explaining every detail of these structures, the script below illustrated the use of the most important of these structures:

```

<?php
//if, elseif and else statements
$a = 2; $b = 3;

if ($a > $b) {
    print "a is bigger than b";
} elseif ($a == $b) {
    print "a is equal to b";
} else {
    print "a is smaller than b";
}
//Would print "a is smaller than b"

//While loop
//Prints the numbers 1 through 10
$i = 1;
while ($i <= 10) {
    print $i++;
    //the printed value would be
    //$i before the increment
    //(post-increment)
}

//For loop
//Prints the numbers 1 through 10
for ($i = 1; $i <= 10; $i++) {
    print $i;
}

//Looping through arrays with foreach
$a = array ("one" => 1,
            "two" => 2,
            "three" => 3,
            "seventeen" => 17);

foreach($a as $k => $v) {
    print "$k: $v.<br>\n";
    //Prints all keys and values
}

//Switch case statement
$i = 1;
switch ($i) {
    case 0:
        print "i equals 0";
        break;
    case 1:
        print "i equals 1";
        break;
}

```

```

        case 2:
            print "i equals 2";
            break;
    }
    //Prints "i equals 1"

?>

```

Use this brief review of the control structures as inspiration and consult the PHP manual for details on syntax and variations of these control structures.

5.4 Functions

Functions are an essential part of any structured programming language. You can think of functions as boxes you can feed values (arguments); the functions process the arguments and return a result to the user (return value). PHP syntax for creating such structures looks like this:

```

<?php

function add_values($num1, $num2) {
    $result = $num1 + $num2;
    return $result;
}

//Print 4
echo add_values(2,2);

?>

```

Arguments and return values can be any of the mentioned data types. You should be aware that global variables (variables declared outside a function) are not defined within a function unless explicitly stated with the `global` keyword:

```

<?php

$num1 = 2; $num2 = 2;

function add_values() {
    //Import globals
    global $num1, $num2;

    //Calculate result
    $result = $num1 + $num2;
    return $result;
}

//Print 4
//Function with no arguments
echo add_values();

?>

```

5.5 Getting more information on the PHP language

With this brief review of the PHP language you should be able to get started and to understand the examples in the remainder of this document. You will however need more information on the PHP language as you move further. The PHP web site (www.php.net) is an excellent resource for documentation, downloads and links.

6 Interfacing databases

The number one reason to use server side scripting is probably to interface databases. There are so many advantages to storing your web content in a database and generate web pages on the fly and most of the programming procedures are quite simple, making it easy to get started. In this section we will review some of the basic techniques for interfacing databases. Most of the basic procedures are the same regardless of platform and database engine, but there are some syntactical differences, and therefore we will review two scenarios: 1) The classic apache web server and MySQL database and 2) the Microsoft way (IIS and Access). But before we come to that we will need to take a look at another programming language used to communicate with databases: Structured Query Language (SQL).

6.1 Structured Query Language SQL

Structured Query Language (SQL) is a universal language used to communicate with databases on most platforms. Different databases have small variations in syntax and the number of available features vary from database to database. The basic part of the language is however the same on all databases.

SQL is used to send queries (instructions and requests) to databases. The commands needed for extracting and inserting information into databases are limited to three commands:

1. SELECT. Used to extract information from the database.
2. INSERT. Create new records in the database.
3. UPDATE. Modify existing records from the database.

It sounds simple and it is! Aside from these functions there are functions for modifying the structure of the database but you rarely need these functions in your PHP scripts. We will review the three basic commands here and later we will add more detail to SQL.

6.1.1 SELECT command

Whenever you need to extract information from a database, you use the **SELECT** command. It has many variation but the basic form looks like this (The code has intentionally been broken into two lines to fit the page, but that is not necessary):

```
SELECT field1, field2, ... FROM tablename
WHERE criteria AND criteria OR criteria
```

It can be extended in many ways to include information from several tables etc. Suppose you have a table in a database looking like this:

ID	name	startprice
1	Michael Schumacher	65
2	Mika Hakkinen	63
3	David Coulthard	50
4	Rubins Barrichello	46
5	Jacques Villeneuve	37
6	Juan-Pablo Montoya	32
7	Ralf Schumacher	40
8	Giancarlo Fisichella	25
9	Oliver Panis	23
10	Heinz-Harald Frentzen	20
11	Jenson Button	18
12	Jarno Trulli	16
13	Nick Heidfeld	11
14	Jean Alesi	10
15	Eddie Irvine	9
16	Pedro de la Rosa	8
17	Luciano Burti	8
18	Kimi Raikkonen	7
19	Jos Verstappen	7
20	Enrique Bernoldi	6
21	Fernando Alonso	5
22	Tarso Marques	4

This is a table with 3 fields: **ID**, **name** and **startprice**. The table is called **drivers**. If we wanted to select everything from this table we would write:

```
SELECT * FROM drivers
```

In many situation we would want to be a bit more specific. For instance we might only be interested in a specific driver:

```
SELECT * FROM drivers WHERE ID = 14
```

This statements produces:

ID	name	startprice
14	Jean Alesi	10

Also we might not be interested in all the fields in the table:

```
SELECT name, startprice FROM drivers WHERE ID < 6
```

This statement produces:

name	startprice
Michael Schumacher	65
Mika Hakkinen	63
David Coulthard	50
Rubins Barrichello	46
Jacques Villeneuve	37

And that is basically what you need to extract basic information from tables. Some of the later examples might be slightly more complicated, but the basic concept is the same.

6.1.2 INSERT command

The INSERT command is used to create new records in the database. The syntax is very simple:

```
INSERT INTO tablename (field1, field2, ...)
VALUES (value1, value2, ...)
```

Using the example from above we could insert a new driver into the table using the following statement:

```
INSERT INTO drivers (name, startprice)
VALUES ('Michael Hansen', 100)
```

Notice that text is enclosed in single quotation marks and also notice that in this case we do not need to fill in all the fields in the table to create a record. The ID field is (in this case) an auto incremental field that automatically creates a new unique ID for every new driver.

6.1.3 UPDATE command

UPDATE is used to change existing fields in existing records. Syntax looks like this:

```
UPDATE tablename SET field1 = value1, field2 = value2
WHERE condition
```

If we for instance wanted to change the name of the driver with ID = 2 we could write:

```
UPDATE drivers SET name = 'Other name' WHERE ID = 2
```

That is basically what you need to know about SQL to get you started.

6.2 Scenario: MySQL database

This section walks you through the basic tasks of extracting, inserting and updating information with PHP and a MySQL database. The examples assumes that you are working on a Unix type system running Apache and MySQL. Most of the example code should however work on other systems as well.

6.2.1 Creating a database and a database user

Before we can manipulate data, we need to create a database and a database user with the proper rights to manipulate the data. If you are already familiar with creating databases and users in MySQL you can skip this section.

First you must log into the database. Type the following on the command prompt:

```
[foo@bar fubaroh]$ mysql -u root -p
Enter password:
```

The login procedure should look something like the example above. You need to know the `root` password for the MySQL server on your system. If it is a newly installed system, the password is empty and you can just press enter to log in. Your screen should look like this:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 23 to server version: 3.23.36
```

Type 'help;' or '\h' for help. Type '\c' to clear the buffer

```
mysql>
```

Start by creating a new database:

```
mysql> create database testdb;
Query OK, 1 row affected (0.16 sec)
```

You then need to start using that database and create a table in the database:

```
mysql> use testdb
Database changed
```

```
mysql> CREATE TABLE people (id int auto_increment primary key,
-> name varchar(255), age int);
Query OK, 0 rows affected (0.06 sec)
```

This means that we create a table with three fields:

1. **id:** Integer value. The field is an auto incremental field and it is also the primary key of the table. Two records can't have the same id number.
2. **name:** A text field containing up to 255 characters.
3. **age:** An integer value.

You could test the table by inserting some values into it:

```
mysql> INSERT INTO people (name, age) VALUES ('Michael', 88);
Query OK, 1 row affected (0.00 sec)
```

Then check if the information is in the table:

```
mysql> SELECT * FROM people;
+-----+-----+-----+
| id | name   | age |
+-----+-----+-----+
|  1 | Michael |  88 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Notice that the `id` field has been set automatically to '1'.

The last thing we need to do is to create a user for access to the database. When connecting to the database from a PHP script, we need to specify a user. If we use the `root` user, the script will in principle have unlimited rights. As this represents a security risk we will create a user with limited rights and use that user for access to the database.

The MySQL database has a privilege system. The system allows you to specify valid users and furthermore to specify what databases, tables and columns the user have access to. You should consult the MySQL manual [5] for details on the privileges system. The following illustrates how to create a user:

```
mysql> use mysql
Database changed
```

```
mysql> INSERT INTO user (Host, User, Password) VALUES
-> ('localhost', 'testuser', password('testpass'));
Query OK, 1 row affected (0.00 sec)
```

This creates a user with no rights. Notice the use of the `password` function. This function creates an encrypted password. The user is allowed to log into the database server but is not allowed to do anything. To grant the user some rights we insert a record in the `db` table:

```
mysql> INSERT INTO db (Host, Db, User,
-> Select_priv, Insert_priv, Update_priv, Delete_priv)
-> VALUES ('localhost', 'testdb', 'testuser',
-> 'Y', 'Y', 'Y', 'Y');
Query OK, 1 row affected (0.04 sec)
```

This allows the user to use the database and specifically the user is allowed to `SELECT`, `INSERT`, `UPDATE` and `DELETE` records. To make the rights take effect, the database server must reload the privilege tables. This is done with the command:

```
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

Now try to log out of the database with the command `quit`. You should then be able to log back into the database with the new user and insert a record in the `people` table:

```
mysql -u testuser -p testdb
```

Now try inserting the record:

```
mysql> INSERT INTO people (name, age) VALUES ('Peter', 34);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT * FROM people;
```

```
+-----+-----+-----+
| id | name   | age |
+-----+-----+-----+
|  1 | Michael |  88 |
|  2 | Peter   |  34 |
+-----+-----+-----+
```

```
2 rows in set (0.06 sec)
```

If you on the other hand try dropping the table (deleting the entire table):

```
mysql> drop table people;
ERROR 1044: Access denied for user:
'testuser@localhost' to database 'testdb'
```

You get an error message because the newly created user is not allowed to remove the table. We now have a new database, a table and a secure user.

6.2.2 Extracting information from the database with PHP

The first thing we want to do with the new database is to extract the information in the table (there are 2 records at the moment), and display them in a table on a web page. We have already seen the SQL statement needed to extract the information, all we need are the PHP commands to send the SQL statement to the database. The procedure for contacting the database and displaying information looks like this (`extract.php`):

```
<html>

<head>
<title>Extracting data</title>
</head>

<body>
<h1>Extracting data</h1>

<table border="1">
<tr>
    <th>ID</th><th>Name</th><th>Age</th></tr>
<tr>

<?php

//Establish link to database server
//$link holds a link to the connection
$link = mysql_connect("localhost","testuser","testpass");

//Select a database
mysql_select_db("testdb",$link);
```

```

//Create a variable holding
//the SQL statement.
$sql = "SELECT * FROM people";

//Send statement to database
//Store link to result
$result = mysql_query($sql);

//Extract one line at a time into an array
//and write HTML
while($record = mysql_fetch_array($result)) {
    echo "<tr>";
    echo "<td>" . $record["id"] . "</td>";
    echo "<td>" . $record["name"] . "</td>";
    echo "<td>" . $record["age"] . "</td></tr>\n";
}

//Close link to database
mysql_close($link);
?>
</table>

</html>

```

The example above can be used as a model whenever you need to send **SELECT** queries to the database.

6.2.3 Creating new records with PHP

The next thing to do is to create an input form for creating new records (create.html):

```

<html>

</head>
<title>Create new record</title>
</head>

<body>
<h1>Create new record</h1>
<form action="create.php" method="post">
Name:<br>
<input type="text" name="name">
<p>
Age:<br>
<input type="text" name="age" size="2">
<p>
<input type="submit"> <input type="reset">

```

```
</form>
```

```
</body>
```

```
</html>
```

Notice that the form sends the values from the form to a script called `create.php`. With our knowledge of SQL and how values are passed from forms to scripts, it is easy to implement the `create.php` script:

```
<html>
<head>
<title>Execute INSERT statement</title>
</head>

<body>
<h1>Execute INSERT statement</h1>

<?php
$link = mysql_connect("localhost","testuser","testpass");
mysql_select_db("testdb",$link);

$sql = "INSERT INTO people (name, age) VALUES ('$name', $age)";
mysql_query($sql);

echo "Attempted SQL statement: <br>$sql";
if (!mysql_error($link)) {
    echo "<p><b>Data inserted</b>";
} else {
    echo mysql_error($link);
}

mysql_close($link);
?>

</body>
</html>
```

This script follows almost the same model as the script for extracting data. The only real difference is the SQL statement. In this case it is an `INSERT` statement instead of `SELECT`. A thing to notice is the primitive error handling. The script uses the function `mysql_error()` to check if the SQL statement generated any errors.

6.2.4 Updating records with PHP

It is a natural thing to implement an editor for the information already present in the database. We will create a script that will display a certain record and save any changes made to the record. The script should determine which record to display from a query string in the URL. If we want to edit the record with `id = 2`, we could type the following URL in the browser:

`http://www.someserver.com/somefolder/editor.php?id=2`

Parameters passed in query strings like above are available in the PHP scripts as variables just like the parameters passed with the `post` method from forms. The script `editor.php` would, in the case above, have a variable called `$id` with the value 2. This makes it very easy to create an editor for the records. We will design the editor in such a way, that it returns to the extraction script (the list of data in the table) when the changes have been saved:

```
<?php

//If the id parameter is not set
//Redirect the browser
if (empty($id)) {
    header("Location: extract.php");
    exit;
}

$link = mysql_connect("localhost","testuser","testpass");
mysql_select_db("testdb",$link);

//If the parameter $change is recieved
//Update database.
//Change is sent from the form on this page
if (!empty($change)) {
    $sql = "UPDATE people SET name = '$name'";
    $sql .= " age = $age WHERE id = $id";
    mysql_query($sql);
    mysql_close($link);
    header("Location: extract.php");
    exit;
}

$sql = "SELECT name, age FROM people WHERE id = $id";
$result = mysql_query($sql);
$person = mysql_fetch_array($result);
mysql_close($link);

?>

<html>
<head>
<title>Editor</title>
</head>

<body>
<h1>Editor</h1>
Editing record number: <?= $id ?> <br>
<p>
<form action="editor.php" method="post">
Name:<br>
```

```

<input type="text" name="name" value="<?= $person["name"] ?>">
<p>
<input type="text" name="age"
    size="2" value="<?= $person["age"] ?>">

<p>
<input type="submit"> <input type="reset">
<input type="hidden" name="id" value="<?= $id ?>">
<input type="hidden" name="change" value="1">
</form>
</body>

</html>

```

Although the database manipulation in this script is rather primitive, it does contain a few new interesting features:

- The first thing to notice is the check to see if there is a value in the parameter `$id`. If not, the user's browser is directed to another page by sending a header field instead of a web page. The `Location` header field causes the browser to be redirected to another page. The `header()` function should be used before any HTML in the document. The HTML causes the standard headers fields to be sent, and this prevents the `header()` function from working properly.
- There is also a check to see if the field variable named `$change` is defined. This variable corresponds to a field in the form generated by the page if `$change` is not defined. The field in the form is of type `hidden`. This means that the field will not be visible to the user (unless he looks at the source code) but the value will be sent. This is an easy way to send values to scripts without the user knowing it.
- The script takes advantage of a quick way to escape HTML mode and print the value of a single expression in the document. The syntax `<?= $id ?>` makes the script print the value of the `$id` variable in the resulting HTML document. It is not particularly efficient to escape HTML mode multiple times, but it can be very advantageous in situation such as the one above.

6.2.5 Finishing the application

One last thing to do, is to link each of the entries in the extraction table to the editor script allowing easy access to the editor from the list of the data. This could be done by changing the extraction script to:

```

<html>

<head>
<title>Extracting data</title>
</head>

<body>

```

```

<h1>Extracting data</h1>

<table border="1">
<tr>
    <th>ID</th><th>Name</th><th>Age</th></tr>
<tr>

<?php

//Establish link to database server
//$link holds a link to the connection
$link = mysql_connect("localhost","testuser","testpass");

//Select a database
mysql_select_db("testdb",$link);

//Create a variable holding
//the SQL statement.
$sql = "SELECT * FROM people";

//Send statement to database
//Store link to result
$result = mysql_query($sql);

//Extract one line at a time into an array
//and write HTML
while($record = mysql_fetch_array($result)) {
echo "<tr>";
echo "<td>" . $record["id"] . "</td>";
echo "<td><a href=\"editor.php?id=". $record["id"] . "\">";
echo $record["name"] . "</a></td>";
echo "<td>" . $record["age"] . "</td></tr>\n";
}

//Close link to database
mysql_close($link);
?>
</table>

</html>

```

This concludes the tour of the basic MySQL facilities in PHP. There are many other available database functions. Consult the PHP manual.

6.3 Scenario: Access database on Microsoft Windows 2000 Server

As already mentioned in section 3.2.3, PHP can only access a Microsoft Access database through an ODBC source, so the syntax for calls to the Microsoft

Access database is general for all ODBC calls. This means, that if you wish to use another database than MySQL or Microsoft Access, the syntax need not be changed at all as long as your database system has an ODBC driver.

There will be no explanation on creating and editing databases with Microsoft Access in this manual.

6.3.1 Extracting information from the Microsoft Access database with PHP

First of all, you have to setup the database and create an ODBC source with a DSN name that you can use to access the database (for details see section 3.2.3). In our examples below, it is assumed that you have Microsoft Access available to create and manipulate a database.

Create a database called `your_user_name.mdb` and make a table called `people` with the columns `name` and `age`. When asked for it, create a primary key. Fill in some rows of test data, and save the database. Remember to change the rights for the file. Now create the ODBC with `testdb` as your systemDSN and fill in the `your_user_name.mdb` as the filename. Now your database is ready for our PHP examples.

The first thing we want to do with the new database is to extract the information in the table, and display it in a table on a web page. Since the call to the ODBC source is general SQL, you already know how to make the query string. The difference between MySQL and the ODBC call is not the query string, but the PHP commands. We have made the same example as in the MySQL scenario, so it is easy to see the similarities and differences between using MySQL and using an ODBC source.

The procedure for contacting the database and displaying information looks like this:

```
<html>

<head>
<title>Extracting data</title>
</head>

<body>
<h1>Extracting data</h1>

<table border="1">
<tr>
    <th>ID</th><th>Name</th><th>Age</th></tr>
<tr>

<?php

//Establish link to database - notice that we establish the link
//directly to the database.
//$link holds a link to the connection
$link = odbc_connect("testdb","", "")
// where testdb is the DSN name
```

```

//Notice that no username and password is supplied
//Access does not offer this kind of protection

//Create a variable holding
//the SQL statement.
$sql = "SELECT * FROM people";

//Send statement to database
//Store link to result
$result = odbc_exec($link, $sql);

//Extract one line at a time and store it in $record
//extract the different fields with odbc_result
//and write HTML
while(odbc_fetch_row($result)) {
    echo "<tr>";
    echo "<td>" . odbc_result($result,"id") . "</td>";
    echo "<td>" . odbc_result($result,"name") . "</td>";
    echo "<td>" . odbc_result($result,"age") . "</td></tr>\n";
}

//Close link to database
odbc_close($link);
?>
</table>

</html>

```

The example above can be used as a model whenever you need to send **SELECT** queries to the database.

6.3.2 Creating new records with PHP

The next thing to do is to create an input form for creating new records:

```

<html>

</head>
<title>Create new record</title>
</head>

<body>
<h1>Create new record</h1>
<form action="create.php" method="post">
Name:<br>
<input type="text" name="name">
<p>
Age:<br>
<input type="text" name="age" size="2">
<p>

```

```

<input type="submit"> <input type="reset">

</form>

</body>

</html>

```

Notice that the form sends the values from the form to a script called `create.php`. With our knowledge of SQL and how values are passed from forms to scripts, it is easy to implement the `create.php` script:

```

<html>
<head>
<title>Execute INSERT statement</title>
</head>

<body>
<h1>Execute INSERT statement</h1>

<?php
$link = odbc_connect("testdb","","");

$sql = "INSERT INTO people (name, age) VALUES ('$name', $age)";
odbc_exec($link, $sql);

echo "Attempted SQL statement: <br>$sql";
if (!odbc_error($link)) {
    echo "<p><b>Data inserted</b>";
} else {
    echo odbc_error($link);
}

odbc_close($link);
?>

</body>
</html>

```

This script follows almost the same model as the script for extracting data. The only real difference is the SQL statement. In this case it is an `INSERT` statement instead of `SELECT`. A thing to notice is the primitive error handling. The script uses the function `odbc_error()` to check if the SQL statement generated any errors.

6.3.3 Updating records with PHP

It is a natural thing to implement an editor for the information already present in the database. We will create a script that will display a certain record and save any changes made to the record. The script should determine which record

to display from a query string in the URL. If we want to edit the record with `id = 2`, we could type the following URL in the browser:

`http://www.someserver.com/somefolder/editor.php?id=2`

Parameters passed in query strings like above are available in the PHP scripts as variables just like the parameters passed with the `post` method from forms. The script `editor.php` would, in the case above, have a variable called `$id` with the value 2. This makes it very easy to create an editor for the records. We will design the editor in such a way, that it returns to the extraction script (the list of data in the table) when the changes have been saved:

```
<?php

//If the id parameter is not set
//Redirect the browser
if (empty($id)) {
    header("Location: extract.php");
    exit;
}

$link = odbc_connect("testdb","","");

//If the parameter $change is recieved
//Update database.
//Change is sent from the form on this page
if (!empty($change)) {
    $sql = "UPDATE people SET name = '$name'";
    $sql .= " age = $age WHERE id = $id";
    odbc_exec($link,$sql);
    odbc_close($link);
    header("Location: extract.php");
    exit;
}

$sql = "SELECT name, age FROM people WHERE id = $id";
$result = odbc_exec($link,$sql);
$person = array("name" => odbc_result($result, "name"),
    "age" => odbc_result($result, "age");
odbc_close($link);

?>

<html>
<head>
<title>Editor</title>
</head>

<body>
<h1>Editor</h1>
Editing record number: <?= $id ?> <br>
```

```

<p>
<form action="editor.php" method="post">
Name:<br>
<input type="text" name="name" value="<?= $person["name"] ?>">
<p>
<input type="text" name="age"
    size="2" value="<?= $person["age"] ?>">

<p>
<input type="submit"> <input type="reset">
<input type="hidden" name="id" value="<?= $id ?>">
<input type="hidden" name="change" value="1">
</form>
</body>

</html>

```

Although the database manipulation in this script is rather primitive, it does contain a few new interesting features:

- The first thing to notice is the check to see if there is a value in the parameter `$id`. If not, the user's browser is directed to another page by sending a header field instead of a web page. The `Location` header field causes the browser to be redirected to another page. The `header()` function should be used before any HTML in the document. The HTML causes the standard headers fields to be sent, and this prevents the `header()` function from working properly.
- There is also a check to see if the field variable named `$change` is defined. This variable corresponds to a field in the form generated by the page if `$change` is not defined. The field in the form is of type `hidden`. This means that the field will not be visible to the user (unless he looks at the source code) but the value will be sent. This is an easy way to send values to scripts without the user knowing it.
- The script takes advantage of a quick way to escape HTML mode and print the value of a single expression in the document. The syntax `<?= $id ?>` makes the script print the value of the `$id` variable in the resulting HTML document. It is not particularly efficient to escape HTML mode multiple times, but it can be very advantageous in situation such as the one above.

6.3.4 Finishing the application

One last thing to do, link each of the entries in the extraction table to the editor script allowing easy access to the editor from the list of the data. This could be done by changing the extraction script to:

```

<html>

<head>
<title>Extracting data</title>

```

```

</head>

<body>
<h1>Extracting data</h1>

<table border="1">
<tr>
    <th>ID</th><th>Name</th><th>Age</th></tr>
<tr>

<?php

//Establish link to database
$link = odbc_connect("testdb","","");

//Create a variable holding
//the SQL statement.
$sql = "SELECT * FROM people";

//Send statement to database
//Store link to result
$result = odbc_exec($link, $sql);

//Extract one line at a time
//extract each field with odbc_result
//and write HTML
while(odbc_fetch_row($result)) {
echo "<tr>";
echo "<td>" . odbc_result($result, "id") . "</td>";
echo "<td><a href=\"editor.php?id="
        . odbc_result($result, "id") . "\">";
echo odbc_result($result, "name") . "</a></td>";
echo "<td>" . odbc_result($result,"age") . "</td></tr>\n";
}

//Close link to database
odbc_close($link);
?>
</table>

</html>

```

This concludes the tour of the basic ODBC facilities in PHP. There are many other available database functions. Consult the PHP manual.

References

- [1] *PHP manual*. Url: <http://www.php.net/manual>. Complete online reference with comments from users.

- [2] Michael Schacht Hansen. *A short introduction to HTML and CSS*. Url: <http://www.intermed.dk/php>.
- [3] Gnu emacs. Url <http://www.gnu.org/software/emacs>. A free, multipurpose texteditor. Features highlighting for several programming languages.
- [4] Ultraedit. Url <http://www.ultraedit.com>. A shareware, multipurpose texteditor. Only available for Windows.
- [5] *MySQL manual*. Url: <http://www.mysql.com/doc/>. Complete online reference with comments from users.